

Global Solution 2026.1

Cibersegurança Cognitiva

Soluções Cognitivas para a Nova Economia Espacial

Solução: Gêmeo Digital para Irrigação de Precisão
Curso: Tecnologia em Inteligência Artificial
Disciplina: Cibersegurança Cognitiva

Conteúdo do relatório: descrição da solução, análise do ambiente de rede, uso de OSINT, automação em Python, identificação de riscos digitais e mecanismos básicos de proteção de dados (hash e criptografia).

Descrição da Solução

A solução é um **Gêmeo Digital para irrigação de precisão** voltado à agricultura. O sistema combina leituras de sensores IoT instalados na lavoura (umidade e temperatura do solo, divididos por quadrantes) com dados climáticos obtidos de uma fonte pública. Um agente inteligente cruza essas informações e decide, para cada quadrante, se a irrigação deve ser ligada, mantida desligada ou se deve aguardar, gerando alertas e registrando tudo de forma auditável.

O objetivo de cibersegurança é demonstrar como os dados trafegam, quais riscos digitais existem e como mecanismos básicos de proteção (hash e criptografia) reduzem esses riscos.

Parte 1 — Como os Dados Circulam na Solução

Origem dos dados

- **Sensores IoT de campo:** umidade e temperatura do solo por quadrante (Q1 a Q4).
- **API pública de clima (Open-Meteo):** temperatura do ar, umidade relativa, precipitação e vento.

Como os dados chegam à aplicação

- Sensores IoT → gateway local → aplicação em Python (aqui simulados).
- Clima → requisição **HTTPS** à API Open-Meteo → resposta em **JSON** → processamento.

Protocolos utilizados

- **HTTPS (HTTP sobre TLS):** usado na chamada da API, garantindo tráfego climático criptografado em trânsito.
- **TCP/IP:** base do transporte de rede entre sensores, gateway e aplicação.
- **JSON:** formato de troca e armazenamento dos dados.

Componentes (APIs, sensores, dados, nuvem)

API externa pública (Open-Meteo) · sensores IoT de solo · agente de decisão em Python · armazenamento local em JSON/CSV · hash SHA-256 para auditoria de integridade.

Parte 2 — Uso de OSINT

Fonte utilizada: API pública **Open-Meteo** (open-meteo.com), que fornece dados meteorológicos abertos, sem necessidade de chave de autenticação — caracterizando uma fonte de inteligência de fontes abertas (OSINT).

Dados obtidos: temperatura do ar (°C), umidade relativa (%), precipitação (mm) e velocidade do vento (km/h) para uma coordenada agrícola de exemplo (lat -23.55, lon -46.63).

Como o Python ajudou na coleta:

```
url = f"https://api.open-meteo.com/v1/forecast?latitude={LAT}&longitude={LON}"
&current=temperature_2m,relative_humidity_2m,precipitation,wind_speed_10m
with urllib.request.urlopen(url, timeout=8) as resp:
    dados = json.loads(resp.read().decode())
```

O Python realiza a requisição HTTPS, recebe o JSON, extrai os campos relevantes e os disponibiliza ao agente de decisão. Há um *fallback* simulado caso não exista conexão, garantindo que a automação rode em qualquer ambiente de avaliação.

Parte 3 — Automação Simples em Python

O script `automacao_irrigacao.py` implementa uma automação completa que cobre vários exemplos pedidos de uma só vez:

- **Coleta de dados de uma API** (Open-Meteo via HTTPS).
- **Leitura de sensores IoT simulados** por quadrante da lavoura.
- **Análise simples**: o agente decide a ação de cada pivô com base em regras (solo seco + sem chuva + ar seco → LIGAR).
- **Salvamento em JSON e CSV** (`registro_irrigacao.json` e `comandos_pivos.csv`).
- **Geração de alerta** para quadrantes que precisam de irrigação imediata.
- **Hash SHA-256** do registro completo para auditoria de integridade.

Exemplo de saída da decisão do agente:

```
[LIGAR] Q2-NorteB -> LIGAR | Solo seco (34.2%), sem chuva, ar seco  
[...]   Q3-SulA  -> AGUARDAR | Condicao intermediaria, monitorando  
ALERTA: Irrigar Q2-NorteB imediatamente!
```

Parte 4 — Identificação de Riscos de Segurança

Foram identificados riscos digitais relacionados à solução. Para cada um, descreve-se o problema, o impacto possível e a forma de mitigação.

Risco	Problema	Impacto	Mitigação
Interceptação de dados (MITM)	Comandos e leituras trafegam pela rede e podem ser interceptados sem criptografia.	Atacante lê/altera comandos, liga ou desliga irrigação indevidamente; perda de safra ou de água.	HTTPS/TLS no tráfego + criptografia simétrica (Fernet/AES) nos comandos.
Manipulação / adulteração de dados	Leituras de sensores podem ser falsificadas, levando o agente a decisões erradas.	Irrigação incorreta (solo úmido irrigado ou solo seco ignorado); degradação da lavoura.	Hash SHA-256 por registro, validando integridade antes de processar.
Exposição de IoT / falha em API (DDoS)	Sensores e API pública expostos; sem rate limiting ou com senha padrão viram vetor de ataque.	Indisponibilidade do serviço, sobrecarga de sensores e acesso não autorizado aos pivôs.	Autenticação por dispositivo, RSA para troca de segredos, rate limiting e segmentação de rede.

Parte 5 — Demonstração de Proteção Básica de Dados

O script `protecao_dados.py` demonstra três mecanismos, cada um ligado ao risco que mitiga:

5.1 — Hash SHA-256 (integridade)

Gera o hash de uma leitura de sensor e, ao adulterar o valor de umidade, o hash muda completamente — comprovando a detecção de adulteração (mitiga o risco de manipulação).

```
Original : 7b15f1375d10ac2d...4613caab9794a
Adulterado: 583ac4f2e7c38c92...cede67ccb79d09 → adulteração detectada
```

5.2 — Criptografia simétrica (Fernet / AES-128)

Uma única chave cifra e decifra os comandos enviados aos pivôs. É rápida e adequada para comunicação frequente entre a central e os dispositivos (mitiga interceptação).

5.3 — Criptografia assimétrica (RSA-2048)

Par de chaves pública/privada: a chave pública pode ser distribuída livremente e cifra a mensagem; apenas a chave privada consegue decifrá-la. Ideal para troca segura de segredos entre componentes (mitiga exposição na distribuição de chaves).

Entregáveis e Critérios de Avaliação

Critério	Pontos	Onde é atendido
Explicação da arquitetura e rede	2.0	Parte 1 (relatório + função <code>descrever_arquitetura_rede</code>).

Uso de OSINT	1.5	Parte 2 (API pública Open-Meteo via HTTPS).
Automação em Python	2.0	Parte 3 (automacao_irrigacao.py).
Identificação de riscos digitais	2.5	Parte 4 (3 riscos com problema/impacto/mitigação).
Uso de hash e criptografia	2.0	Parte 5 (protecao_dados.py: SHA-256, Fernet, RSA).

Código-fonte: protecao_dados.py e automacao_irrigacao.py (anexos). **Vídeo:** roteiro de 5 a 10 minutos entregue em documento separado.